

Group Members:

Leul Tamiru - u25270665

Frank Yambayamba - u25076427

Shakir Alexander - u25122208

N'des Junior Lungwangu - u2506936

Kadin Raju - u25175956

Onalerena Mojela - u25021002

Thakasa Ngcobo - u25556526

Task 1

1. What is a workflow and what is a workflow management system?
 - A workflow is a structured sequence of activities that transforms and input into an outcome, defining what work must be done, who performs it, in what order and what happens when it succeeds, fails or if it requires escalation. A workflow management system is a software that stores workflow definitions and uses them to create and manage many running instances of that process, coordinating whichever of work items, participants, routing, approvals, events, integration, history and reporting the process. **JIRA** is a practical example, letting teams define customizable workflows suited for their own needs. (Atlassian, n.d.)
2. How are activities or work items represented?
 - Across multiple workflow managing platforms, activities and or work is represented as a table that looks like an Excel spreadsheet. In the table, common categories like a summary of the task, the status (TO-DO), a due date, the priority of the task and comments that may have been left by other teammates. These are the most important things a user needs to know about the task.
3. How are work items assigned and routed?
 - Tasks are directly assigned to users based on their user ID (Oracle, n.d.). Tasks can be shared to a queue or to a specific role rather than a single person, any member that has that specific role can either claim or be assigned to a task. Tasks can also be assigned conditionally, factors like priority, budget, amount, department or even geo-location can decide who receives the task. They are usually routed in the following 3 ways, **sequential** - here tasks move in a strict linear chain where completion of one task immediately triggers a route to the next tasks in line, **parallel** – a single step branches out into multiple simultaneous tasks assigned to different people or departments at the same time, **conditional** – the path

of workflow changes based on an outcome or decision and finally, **escalation & fallback** – if a task expires of the designated user isn't available, the workflow manager automatically re-routes or escalates the item to a supervisor or an alternative handler.

4. What is the difference between a workflow definition and a running workflow instance?
 - A workflow definition is the reusable template for a process, created once and describing its stages and legal transitions, while a workflow instance is one live, independent execution of that template, with its own participants, data and progress. Editing or deleting a running instance has no effect on the definition it was created from, and the same definition can spawn many instances running at completely different points in the process simultaneously. In Camunda, this is enforced directly: a deployed process definition is immutable once instances are running against it, and each instance tracks its own variables and current state independently of every other instance (Camunda, n.d.-b). This is the foundational separation TIVIDY is built around, so it needs distinct definition and instance classes from the very start rather than a single class that tries to do both jobs.
5. How are approvals, rejection and escalation handled?
 - Approval steps require one or more designated participants to review and either approve or reject a work item before it proceeds; a rejection typically routes the item back to the submitter or a prior stage rather than stopping the workflow outright. In ServiceNow, approvals can require a single approver, a unanimous group, or majority voting, and the system automatically escalates to a backup approver if a deadline passes without a decision (ServiceNow, n.d.). This pattern repeats across most of the platforms researched, so TIVIDY needs configurable approval logic rather than one hardcoded approve/reject check.
6. How are participants, roles and responsibilities represented?
 - Participants are usually represented separately from the tasks they perform, as users, groups, or roles that work items get assigned to, rather than named individuals hardcoded into the process itself. Camunda models this with candidate groups and candidate users, so a task can be claimed by anyone holding a given role instead of needing to know in advance who specifically will handle it (Camunda, n.d.-a). Since responsibilities shift as organisations reorganise, representing participants abstractly by role needs to be a core part of TIVIDY's design rather than fixed inside individual tasks.
7. How are dependencies, events and failures handled?
 - Dependencies are usually modelled as explicit links between tasks that control ordering, a task cannot start until its prerequisite finishes, while events represent things that happen during execution, a timer expiring, a message arriving, an error being thrown, that can start, interrupt or redirect a process. BPMN, used by engines like Camunda and jBPM, defines dedicated timer, error and message event types specifically so failures and triggers can be modelled directly rather than hidden inside

code (OMG, 2013). This suggests TIVIDY needs events treated as first-class objects the engine can react to, not exceptions caught ad hoc.

8. How do workflow systems interact with external systems?
 - Workflow systems typically expose integration points, often called connectors, service tasks or webhooks, that let a running process call out to an external system, sending an email, charging a payment, updating a separate database, partway through execution. Microsoft Power Automate works almost entirely around pre-built connectors to hundreds of third-party services, treating each integration as a swappable unit rather than special-cased code inside the workflow engine (Microsoft, n.d.). This is a strong signal that TIVIDY should isolate integration logic behind a common interface, since the specific external systems involved will vary per deployment.
9. How is workflow history or audit information recorded?
 - Most systems log every state change, assignment and decision made during a workflow instance's life as an append-only, timestamped audit trail tied to whichever user or system caused it, so the process's history can be reconstructed later for compliance or debugging. SAP Business Workflow keeps a dedicated work item history log separate from the live process data, specifically so audits do not touch or risk corrupting the running instance (SAP, n.d.). This suggests TIVIDY should treat history as a record attached to each instance, decoupled from the instance's current working state.
10. How are running workflows monitored and reported on?
 - Monitoring tools typically track running instances in aggregate, how many are active, how long tasks are taking, where bottlenecks are forming, usually through a separate dashboard or reporting layer rather than functionality built into the core workflow engine itself. Camunda Optimize ships as an entirely separate product from the Camunda engine for this exact reason, so reporting features can evolve independently of the core runtime (Camunda, n.d.-c). This reinforces that TIVIDY's monitoring and reporting responsibility should be a separate, pluggable layer that observes instances rather than logic embedded inside the work-item classes themselves.
11. What makes a workflow engine reusable across different processes?
 - A workflow engine becomes reusable across different processes when the engine itself contains no process-specific logic, routing decisions and approval chains are supplied externally as configuration or pluggable rules rather than hardcoded into the engine's classes. jBPM and Camunda are both marketed as general-purpose engines precisely because the same runtime can execute entirely different process definitions without any change to the engine's source code (Red Hat, n.d.). The implication for TIVIDY is that genericity has to be a deliberate design constraint, since any process-specific assumption baked into the engine breaks reusability.
12. Which information belongs to a workflow definition and which belongs to a running instance?
 - A workflow definition holds everything that describes how a process is allowed to behave, its stages, legal transitions and rules, while a running

instance holds only the data specific to one execution of that process, its current state, participants and progress. In Camunda, a BPMN process definition is deployed once and referenced by many separate process instances, each with its own variables and execution state, without any instance ever modifying the shared definition (Camunda, n.d.-b). This separation is consistent enough across the research that TIVIDY needs genuinely distinct class hierarchies for definitions and instances, not a single class trying to do both jobs.

13. How do real workflow systems model nested activities or sub-processes?

- Real workflow systems typically model a sub-process as a self-contained process definition that can be invoked from within a parent process, treated by the engine as a single step from the parent's perspective but internally made up of its own stages and transitions. BPMN explicitly supports call activities and embedded sub-processes for this reason, letting a large process be broken into smaller, independently testable pieces instead of one flat, unmanageable diagram (OMG, 2013). This points to TIVIDY needing a composite-style structure, where a single unit of work and a group of work can both be treated uniformly by the rest of the system.

14. Which decisions are normally configurable rather than hard-coded into the workflow engine?

- Decisions that tend to vary by organisation or change frequently, routing and assignment rules, approval chains, escalation timeouts, which integrations fire on which event, are the ones real systems keep configurable, while the core mechanics of moving a work item through valid states generally stay fixed inside the engine. Appian lets non-developers configure routing and approval rules through a visual designer rather than requiring a code change, specifically because those rules are expected to change far more often than the underlying engine (Appian, n.d.). This is the clearest signal from the research for where TIVIDY needs pluggable, pattern-based extensibility rather than hardcoded logic.

15. How are changes to workflow definitions handled while work is already in progress?

- Most mature workflow systems version their definitions rather than overwriting them, so a running instance keeps executing against the exact version of the definition it started under, while new instances pick up the newest version going forward. Camunda explicitly supports multiple deployed versions of the same process definition running at once for this reason, since forcing an in-flight instance to suddenly follow a changed process could leave it in an invalid or undefined state (Camunda, n.d.-b). This means TIVIDY's definitions cannot be treated as mutable objects instances simply point to, versioning needs to be built in from the start.

References

Appian (n.d.) Route Workflows Automatically. Available at:
<https://docs.appian.com/suite/help/26.6/automate-routing.html> (Accessed: 1 October 2026).

Atlassian (n.d.) Understand workflows. Available at:
<https://support.atlassian.com/jira-cloud-administration/docs/work-with-issue-workflows/>
(Accessed: 1 October 2026).

Camunda (n.d.-a) User tasks. Available at:
<https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/> (Accessed: 1 October 2026).

Camunda (n.d.-b) Versioning process definitions. Available at:
<https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/> (Accessed: 1 October 2026).

Camunda (n.d.-c) Camunda Optimize reports. Available at:
<https://camunda.com/platform/optimize/reports/> (Accessed: 1 October 2026).

Microsoft (n.d.) Connectors overview. Available at:
<https://learn.microsoft.com/en-us/connectors/overview> (Accessed: 1 October 2026).

Object Management Group (2013) Business Process Model and Notation (BPMN), Version 2.0.2. Available at: <https://www.omg.org/spec/BPMN/2.0.2/PDF> (Accessed: 1 October 2026).

Oracle (n.d.) How workflow task routing is mapped out. Available at:
<https://docs.oracle.com/en/cloud/saas/human-resources/faucf/how-workflow-task-routing-is-mapped-out.html> (Accessed: 1 October 2026).

Red Hat (n.d.) jBPM documentation. Available at:
https://docs.jboss.org/jbpm/release/7.42.0.Final/jbpm-docs/html_single/ (Accessed: 1 October 2026).

SAP (n.d.) Workflow audit. Available at:
https://help.sap.com/docs/SAP_DISCLOSURE_MANAGEMENT/6fda503523d6413597f5cd81be59fa8a/495e9a5425504582a3d871cf5a9ec22b.html (Accessed: 1 October 2026).

ServiceNow (n.d.) Simple approvals in workflow. Available at:
https://support.servicenow.com/kb?id=kb_article_view&sysparm_article=KB0538552
(Accessed: 1 October 2026).

Task 2

Initial scope of TIVIDY

- TIVIDY will be designed as a generic workflow system capable of defining and executing several types of workflows. The system should remain independent of any specific business process so that the same workflow can be reused in different environment. This adds onto our research, which shows that reusable workflow engines keep process specific rules separate from the core engine.

Main responsibilities of TIVIDY

- Storing reusable workflow definitions;
 - creating independent workflow instances from those definitions;
 - representing individual units of work as WorkItems;
 - organising work into Stages and nested sub-processes;
 - managing dependencies between WorkItems;
 - supporting both sequential and parallel work;
 - assigning available WorkItems to suitable participants according to configurable rules;
 - managing the lifecycle of WorkItems;
 - applying routing rules to determine which work becomes available next;
 - supporting approvals, rejection and escalation;
 - reacting to internal and external events;
 - communicating with external systems through generic integration interfaces;
 - maintaining workflow history and state changes;
 - supporting recovery or rollback where appropriate; and
 - exposing workflow information for monitoring and reporting.
- create and manage running workflow instances
- Manage the work items and tasks in a workflow
 - Track the progress of work items
 - Control the routing of work from one activity to another
 - Assign work to the correct participants
 - Manages approvals, rejections, failures in the system
 - Communicate with external systems when required
 - Notify interested participants when changes occur
 - Maintain a history of important workflow actions

Boundaries

- TIVIDY is responsible for managing the workflow process, but it should not contain specific details for a particular organisation.

The system should use generic concepts such as:

- Workflow names
- Work items
- Participants
- Roles
- States
- Events
- Rules and actions
- Workflow Definition
- Is the reusable description of how a process is allowed to operate.

It should contain information such as:

- Workflow name
- The stages in the process
- The order in which work occurs
- Valid transitions between stages

- Routing rules
- Assignment rules
- Approval and escalation requirements
- Possible events and failure paths

The definition does not represent one active execution of a process. It acts as a template from which multiple workflow instances may be created.

Running Workflow Instance

- Represents one active execution of a workflow definition.

Each instance should contain its own runtime information:

- A unique instance identifier
- The current workflow state
- The participants assigned to work
- Current progress
- Timestamps
- Failures or exceptions that have occurred
- Execution history

Multiple workflow instance will use the same workflow definition while being at completely different points in the process.

Parts of the system expected to vary

- This depends on the organisation or process being executed.

Includes:

- routing rules
- participant and task assignment rules
- validation rules
- approval requirements
- escalation rules
- actions performed during the workflow
- external systems used
- notification recipients
- execution strategies
- optional behaviour added to work items

Selected Workflow: ArkhamExpress

- Organisation:
 - ArhamExpress is an order fulfilment and delivery organisation responsible for processing customer orders and delivering packages to customers.

- The organisation includes customers, warehouse staff, drivers, and delivery managers.
- ArkhamExpress will use TIVIDY to coordinate the different activities involved in processing and completing a customer delivery.

Process Being Managed

- The fulfilment and delivery of a customer order.
- The workflow begins when a customer places an order and continues through validation, stock checking, package preparation, delivery type delegation, driver assignment and deliver.

The workflow ends when the order reaches the outcome:

- Delivered
- Cancelled
- Returned
- Failed

Participants and Roles

- Customer
 - Places an order
 - Receives notifications regarding order
 - Receives final package
- Warehouse staff
 - Locate required stock
 - Prepare products
 - Prepare package for dispatch
- Driver
 - Receives the delivery assignment
 - Collects the package
 - Completes the deliver
 - Updated delivery status
- Delivery Manager
 - handles exceptional situations
 - approves certain special actions where required
 - assists with failed or escalated deliveries
- TIVIDY
 - manages the workflow
 - routes work
 - tracks the current state

- assigns work
- reacts to workflow events

Major prices of Work

- Receives the customer order
- Validate the order
- Check product availability
- Request missing stock if necessary
- Prepare the package
- Select the required delivery type
- Determine the delivery method
- Assign a driver
- Dispatch the package
- Track the delivery
- Complete the delivery

Additional work may be required if a problem occurs during the process:

- Retrying a delivery
- Rerouting a deliver
- Reassigning a driver
- Returning or cancelling a delivery

Important decisions and Rerouting points

- *Is the order valid?*
 - Yes – the package can be prepared
 - No – the stock must be requested
- *What delivery type is required?*
 - Decision is based on the following factors:
 - Distance
 - Urgency
 - Destination
 - Package type
- *Is the driver available?*
 - Yes – the driver is assigned
 - No – the workflow may wait, reassign work, or escalate the problem
- *Was the delivery successful?*
 - Yes – the workflow is completed
 - No the system may retry, reroute, return or cancel delivery

Approval and Dependencies

- an order must be validated before it can continue
- stock must be available before the package can be prepared

- the package must be prepared before it can be dispatched
- a driver must be assigned before delivery can begin
- the package must be dispatched before the delivery can be completed

These dependencies ensure that work is performed in the correct order. The research identified dependencies as explicit relationships between tasks that prevent later work from starting before required earlier work has completed.

Approval may also be required in certain exceptional cases. For example, a high-value or special delivery may require approval from a Delivery Manager before dispatch. Approval may also be required before certain cancellations, rerouting decisions or special handling actions.

Possible Failures or Exceptions

- Possible failures and exceptional situations include:
 - the order failing validation
 - required stock being unavailable
 - an external stock request failing
 - no driver being available
 - a package being damaged
 - an incorrect delivery address
 - the customer being unavailable
 - a delivery attempt failing
 - an external warehouse or courier system being unavailable
 - a workflow action being cancelled.

External Systems

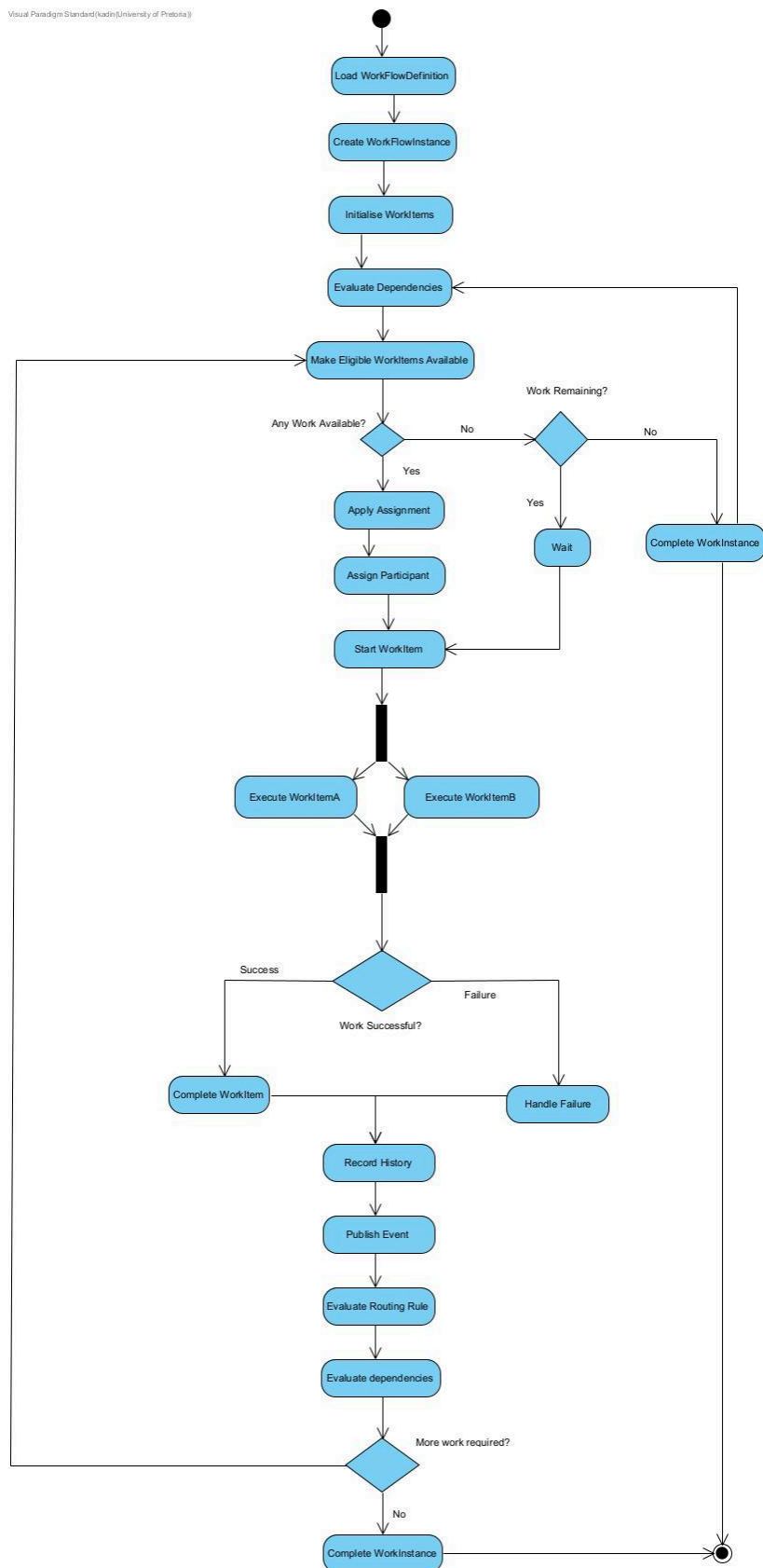
- ArkhamExpress may need to communicate with external systems during workflow execution.
- The primary external system will be an External Warehouse System. This system may be contacted when the required stock is not available locally.

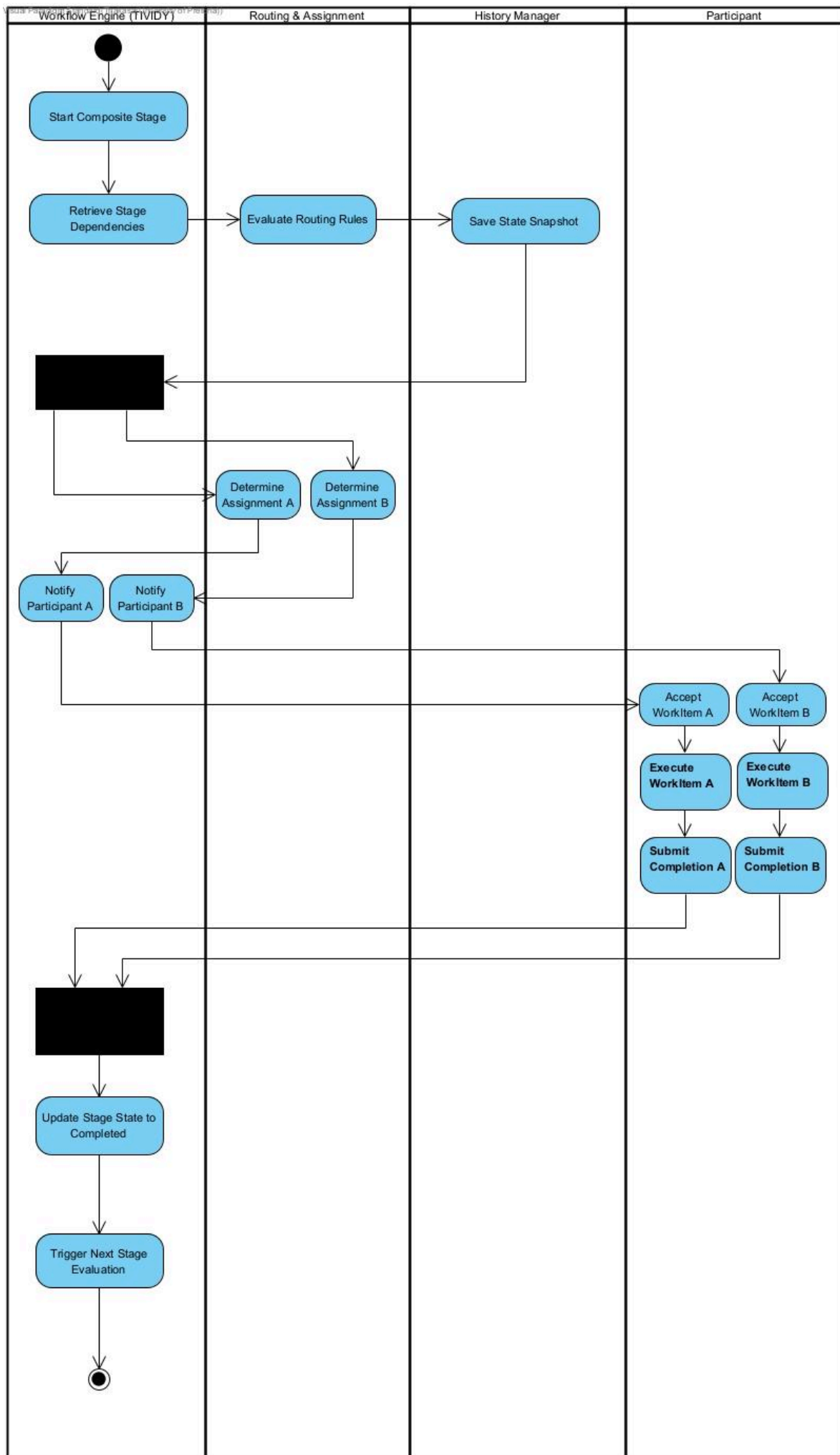
Expected Outcome

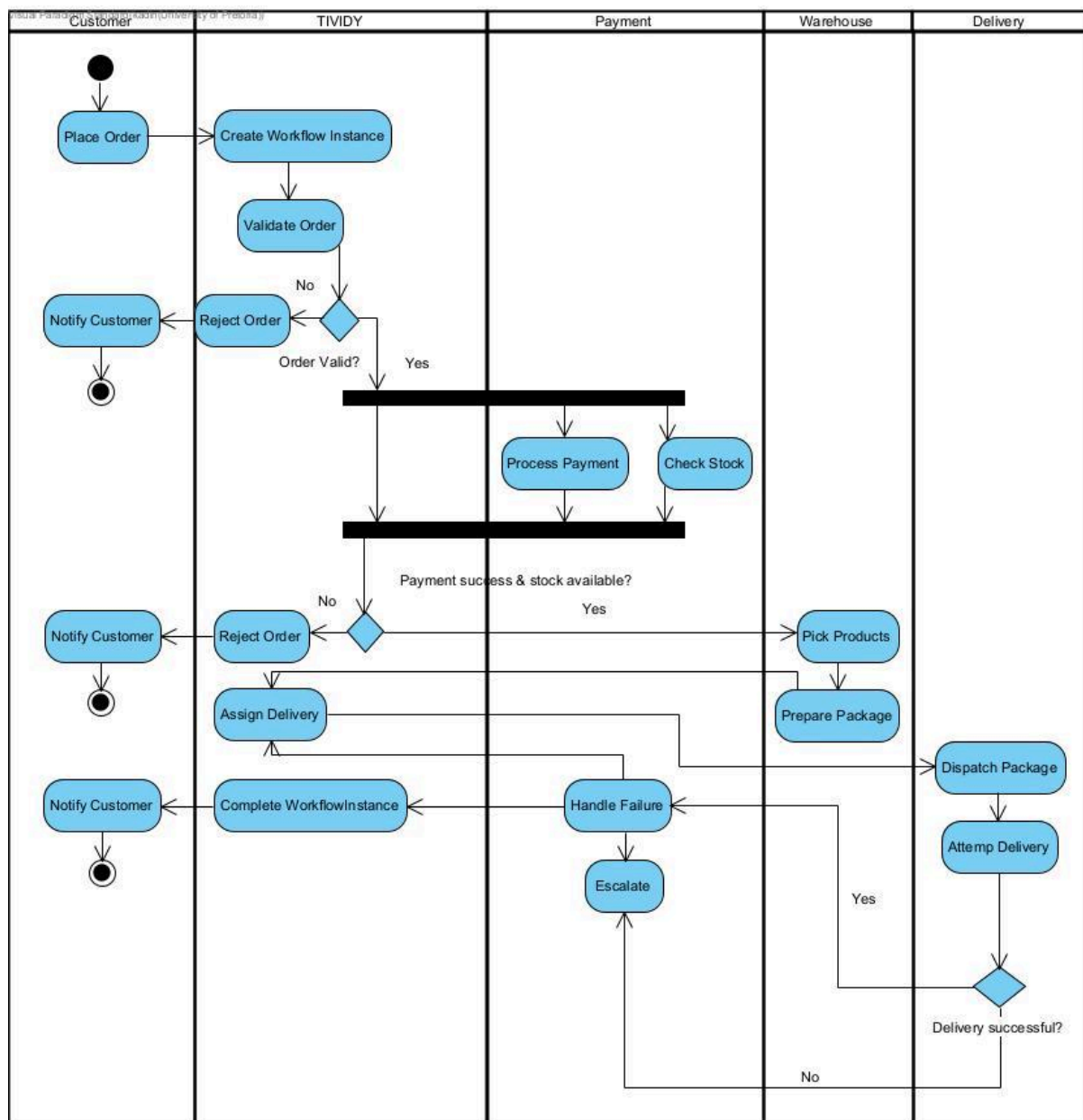
- The main successful outcome is that the customer's order has been:
 - Received
 - validated
 - fulfilled
 - packaged
 - assigned for delivery
 - dispatched
 - successfully delivered

Task 3

Visual Paradigm Standard (vadin@University of Pretoria)







Task 4

Composite – Workflow Structure

- TIVIDY needs to support individual pieces of work, groups of related work and potentially nested processes. The system should be able to work with these structures without needing to know if it's dealing with one task or a group of tasks.

Participants

- Component: WorkflowComponent
- Leaf: WorkItem
- Composite: Stage
- WorkflowComponent provides a common abstraction for individual and grouped work. May contain WorkItems or nested stages. Because both WorkItem and Stage share the same parent abstraction, the workflow can be represented as a tree-like structure.

What varies

- The organisation and nesting of work.

Why Composite

- It allows TIVIDY to represent arbitrary workflow structures without clients needing to understand their internal storage.

State – WorkItem Lifecycle

- A WorkItem changes during the execution of a workflow. Different operations are needed depending on the WorkItem's current lifecycle.
- WorkItems may move through generic states: available, assigned, started, completed, rejected or escalated.

Participants

- Context: WorkItem
- State: WorkItemState
- ConcreteState:
 - CreatedState
 - AvailableState
 - AssignedState
 - StartedState

- CompleteState
- CancelledState

A workItem delegates lifecycle specific operations to its current WorkItemState. Each concrete state decides which operations are legal and what transition should occur.

What varies

- The behaviour allowed at different lifecycle states.

Why State

- It avoids large conditional statements and keeps scenario-specific names such as “Shipped” outside the engine.

Strategy – Routing and Assignment Rules

TIVIDY needs to decide what work comes next and who gets it, but every organisation decides this differently. These decisions should not be written into WorkflowInstance or WorkItem, so the engine doesn't change whenever a process does.

Participants:

- Strategy: RoutingRule (Interface) – decides what work becomes available when a work item finishes.
- Strategy: AssignmentRule (Interface) – decides which Participant an available work item is given to.
- ConcreteStrategy: SequentialRouting, DataBasedRouting, RoleBasedAssignment, WorkloadBasedAssignment
- Context: WorkflowInstance

WorkflowInstance is given its rules by its WorkflowDefinition and only calls the RoutingRule and AssignmentRule interfaces, so it never knows which rule it is using. New rules are added as new ConcreteStrategies without changing the engine.

Collaboration

- The engine delegates a decision to a configured strategy.

What varies

- The decision algorithm.

Why Strategy

- The workflow engine remains unchanged when an organisation changes its assignment or routing policy.

Observer – Events and Reactions

When a WorkItem changes state, several parts of the system may need to react, such as making the next work available, notifying a participant or updating monitoring. The WorkItem should not need to know about any of them.

Participants:

- Subject: EventSource (attach, detach, notify)
- ConcreteSubject: WorkItem
- Observer: EventListener (Interface)
- ConcreteObserver: WorkflowInstance, NotificationObserver, MonitoringObserver

When a WorkItem changes state, it notifies every attached EventListener with an Event. WorkflowInstance reacts by making the next work available, and the other listeners react independently. New listeners can be attached without changing WorkItem.

What varies

- Which components react and how.

Why Observer

- Event producers do not need direct knowledge of all interested components.

Chain of Responsibility – Approval and Escalation

Allows for the system to route a request through multiple levels until someone handles it. The system needs to handle approvals(outcome is yes or no after going through several levels) or escalations (when nobody acts in time and the work is pushed onto the next guy).

Participants:

- Handler: ApprovalHandler (Interface)
- ConcreteHandler: RoleBasedApprover (Checks if the current role has the authority to approve the request, if not it will pass onto the next level).
- Client: WorkItem (Initiates the approval request/escalation)

When a WorkItem needs approval it will be sent to ApprovalHandler in the chain.

What Varies

- The approval levels, who sits at each level and the time limits that each organisation has.

Why is it appropriate

- It allows the system to process approvals and escalations without hardcoding the exact chain or the roles that may be involved. The chain can be configured at runtime.

Memento – History and Rollback

This should allow for the workflow system to hold any state changes and support an undo where necessary.

Participants:

- Originator: WorkflowInstance (Creates the snapshot of its current data and the generic states of its work items)
- Memento: InstanceSnapshot (stores the internal state of WorkflowInstance)
- Caretaker: HistoryManager (Requests and retains the ordered record of actions and state changes on an instance)

Before state change in WorkItem, the HistoryManager will request a snapshot from the WorkflowInstance, obtain a InstanceSnapshot and if a failure occurs in the path, the HistoryManager passes the InstanceSnapshot back to WorkflowInstance to restore its previous state.

What Varies

- The state of the workflow instance over time and the point at which the system may need to return to.

Why is it Appropriate

- Keeps encapsulation of the WorkflowInstance by not exposing its internal variables while supporting history.

Builder – Workflow Construction

Workflow definition tells us what work exists, the order and the rules. This makes it a complex object as you have various work items and sub processes that we will not hard-code to a specific scenario.

Participants:

- Builder: WorkflowBuilder (Declares interface for adding of stages, work items, dependencies and assignment rules)
- ConcreteBuilder: ScenarioBuilder (Implements the steps to assemble a specific process)

- Director: EngineConfigurator (Directs the builder to do the construction in the correct order)
- Product: WorkflowDefinition (The final representation of the process)

The Director will call step-by-step methods on the WorkflowBuilder. The Director will then retrieve the fully assembled WorkflowDefinition.

What Varies

- the specific arrangement of work items, stages and rules supplied by different organizations for different scenarios.

Why it is Appropriate

- Prevents huge, unreadable constructors and separates the assembly logic from a scenario from the generic execution logic of the workflow engine.

Decorator – Additional Behaviours

Allows for additional behaviours or functionality to be added onto already created tasks at runtime.

Participants:

- Component: WorkFlowComponent
- ConcreteComponent: WorkItem
- Decorator: TaskDecorator
- ConcreteDecorator: ValidationDecorator, AuditDecorator, DeadlineDecorator, RetryDecorator, etc

Just like with the **Composite** design pattern, WorkFlowComponent provides a common abstraction for individual or group items and as a result, the workflow can be represented as a tree

What varies

- which behaviours each work item has, which work items get behaviour, the parameters of each behaviour and the order of the stack.

Why Decorator

- We can now add extra functionality to an object at runtime without having to change the original class, everything can get easily added/removed at runtime which meets the requirements of our workflow system

Command

Allows for requests to be turned into objects so that they can be recorded, stored, queued and even undo it.

Participants:

- Command: Command(Abstract)
- ConcreteCommand: AssignCommand, StartCommand, CompleteCommand, RejectCommand, CancelCommand
- Receiver: WorkItem
- Invoker: Workflow instance or the execution engine
- Client: whatever receives a request

What varies

- the operations that are requested, which receiver acts on a work item/instance, the data the command carries and when and who a command is created

Why Command:

- It's the only way to get Action, History and Rollback and it allows for each operation to own its own undo action.

Adapter

Problem

- Work items in a workflow sometimes need an external system to do the work, such as a carrier, a payment provider or a records system. Each of these has its own interface that was not designed for TIVIDY, and we cannot modify it. If engine classes called these interfaces directly, the engine would contain system-specific code, and every new integration or vendor API change would mean editing the engine.

Participants

- Target: Connector, an abstract class owned by the engine, declaring the generic operations the engine uses to invoke any external system.
- Adaptee: the external system's existing interface, for example PostalServiceAPI.
- Adapter: PostalServiceConnector, which inherits from Connector and holds a PostalServiceAPI.

Collaboration

- The client calls an operation on Connector. The adapter translates the request into the adaptee's operations and data format, calls the adaptee, translates the response back, and returns it. The client never knows which external system handled the call.

What varies

- The set of external systems and their interfaces varies between organisations and over time. The Connector interface and the engine code that calls it do not. A new integration is one new adapter class with no changes to the engine, and a vendor API change is confined to that vendor's adapter.

Why Adapter

- The mismatch is between an interface we own and existing interfaces we cannot change, which is the situation Adapter addresses. We use an object adapter so the adapter holds the adaptee rather than inheriting from it, which avoids multiple inheritance and makes ownership of the adaptee explicit.

Visitor – Monitoring and Reporting

- TIVIDY needs to produce views over a running workflow, such as progress, overdue work and audit summaries, and every organisation wants different ones. Each report has to walk the whole tree of stages and work items and do something different for each kind. If every report were an operation on WorkFlowComponent, WorkItem and Stage, the core model would have to be edited each time a new report is needed, and reporting code would sit inside the classes that run the workflow.

Participants:

- Visitor: WorkflowVisitor (Interface) – declares visitWorkItem(WorkItem) and visitStage(Stage).
- ConcreteVisitor: ProgressReportVisitor, OverdueWorkVisitor, AuditReportVisitor
- Element: WorkFlowComponent – declares accept(WorkflowVisitor).
- ConcreteElement: WorkItem, Stage
- ObjectStructure: WorkflowInstance – owns the root of the work tree.
- Client: MonitoringService – creates a visitor and hands it to a WorkflowInstance.

The client passes a ConcreteVisitor to the WorkflowInstance, which calls accept on its root WorkFlowComponent. A WorkItem calls visitWorkItem(this) on the visitor. A Stage calls visitStage(this) and then passes the visitor to each of its children, so the whole tree is covered. A TaskDecorator forwards accept to the work item it wraps. The visitor collects what it needs as it goes, and the client reads the result from the visitor at the end.

What varies

- the set of reports, which is supplied per organisation and grows over time. The kinds of element do not vary: the engine only has work items and stages,

whatever the scenario. A new report is one new ConcreteVisitor with no changes to WorkItem, Stage or WorkflowInstance.

Why Visitor fits

- because the element classes are few and stable while the operations over them keep changing, which is the case the pattern is meant for. It also removes the need for type checks, since each element calls the visit operation for its own type. It keeps monitoring and reporting out of the core model, as the spec asks.

Proxy – Access Control

- Only some participants may see or perform certain operations on work, for example only one role may cancel a work item. This check must happen before the operation reaches the work item, but it should not be written inside WorkItem or repeated in every part of the engine that touches one. Who is allowed to do what also differs for every organisation.

Participants:

- Subject: WorkflowComponent – the interface the engine already uses for work.
- RealSubject: WorkItem, Stage
- Proxy: AccessControlProxy – holds a WorkflowComponent and an AccessPolicy.
- Client: Command objects (AssignCommand, StartCommand, CompleteCommand, RejectCommand, CancelCommand) – call the work through WorkflowComponent.

AccessPolicy (Interface) is a helper that answers whether a Participant, through its Roles, may perform a given operation. It is supplied by the WorkflowDefinition.

The client calls an operation on what it believes is a WorkflowComponent. The AccessControlProxy asks its AccessPolicy whether the requesting Participant is allowed to perform that operation. If yes, the proxy forwards the call to the real WorkItem or Stage and returns the result. If no, the call is refused and the real object is never reached. The client cannot tell whether it holds a proxy or the real object.

What varies

- is the access policy: which roles may perform which operations, supplied per organisation. The work classes, the commands and the proxy itself do not change. Protected and unprotected work can be mixed in the same workflow.

Why Proxy fits

- because the problem is controlling access to an object without changing the object or its clients, which is a protection proxy. It differs from our Decorator: a

decorator always passes the call on and adds behaviour around it, while the proxy decides whether the call happens at all.

Facade

- Intended to provide a unified subsystem of workflow creation, assignment and execution, while hiding the intricate interactions between builders, managers and instances.

Participants

- Facade (WorkflowEngine) - The main entry point for client applications. It provides simple methods like DeployWorkflow(), StartInstance(), GetAvailableWork(), and CompleteWorkItem().
- Subsystems
 - WorkflowBuilder / EngineConfigurator (Builder)
 - WorkflowInstance and HistoryManager (Memento)
 - EventSource (Observer)
- Client (ExternalApplication / ArkhamExpressSystem) - The specific business application that uses TIVIDY to run its processes.

How it works

- When the ArkhamExpress system says "Order Placed", it simply calls WorkflowEngine.StartInstance("OrderDelivery"). The Facade handles the complex delegation of querying the WorkflowDefinition, initializing a new WorkflowInstance, and triggering the first WorkItem state change.

What part of the design is expected to vary

- The specific client applications integrating with TIVIDY (such as ArkhamExpress) and the sequence of internal subsystem interactions (using components like HistoryManager and EventSource) required to perform high-level workflow operations.

Why the selected pattern is appropriate

- It completely decouples external business systems from the complex internal architecture of the workflow engine. By providing a simplified entry point, the engine's internal classes can be independently modified or expanded without breaking the external applications that rely on it to start and manage workflow instances.

Iterator

Provides a way to access the element of a workflow tree sequentially without exposing its internal structure.

Participants:

- Iterator (WorkflowIterator) - Defines standard traversal methods like first(), next(), isDone(), and currentItem().
- ConcreteIterator (AvailableWorkIterator / HistoryIterator) – Implements the traversal logic.
- Aggregate (WorkflowComponent) – defines the method for creating the iterator
- ConcreteAggregate (State / WorkflowInstance) – returns a specific ConcreteIterator

How it works

- Instead of the WorkflowInstance writing a giant loop to check every nested stage for available tasks, it just asks the root Stage for an AvailableWorkIterator. The engine can step through iterator.next() to assign tasks via the AssignmentRule Strategy, completely ignorant of how deeply nested the workflow structure actually is.

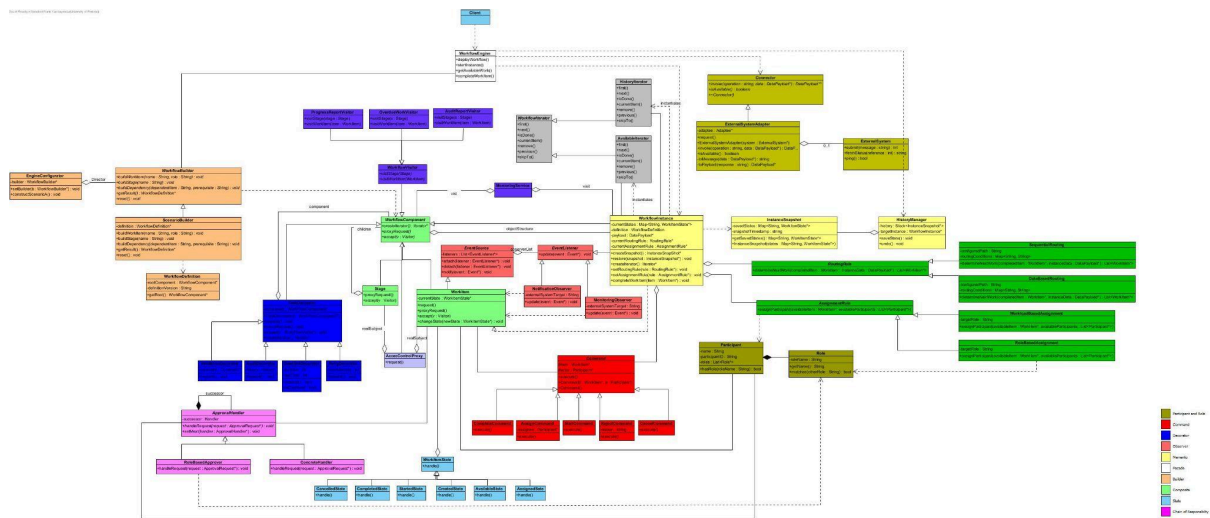
What part of the design is expected to vary

- The traversal algorithms used to navigate the workflow execution (e.g., finding the next available tasks versus auditing failed tasks) and the internal tree-like structure of nested WorkflowComponent and Stage objects.

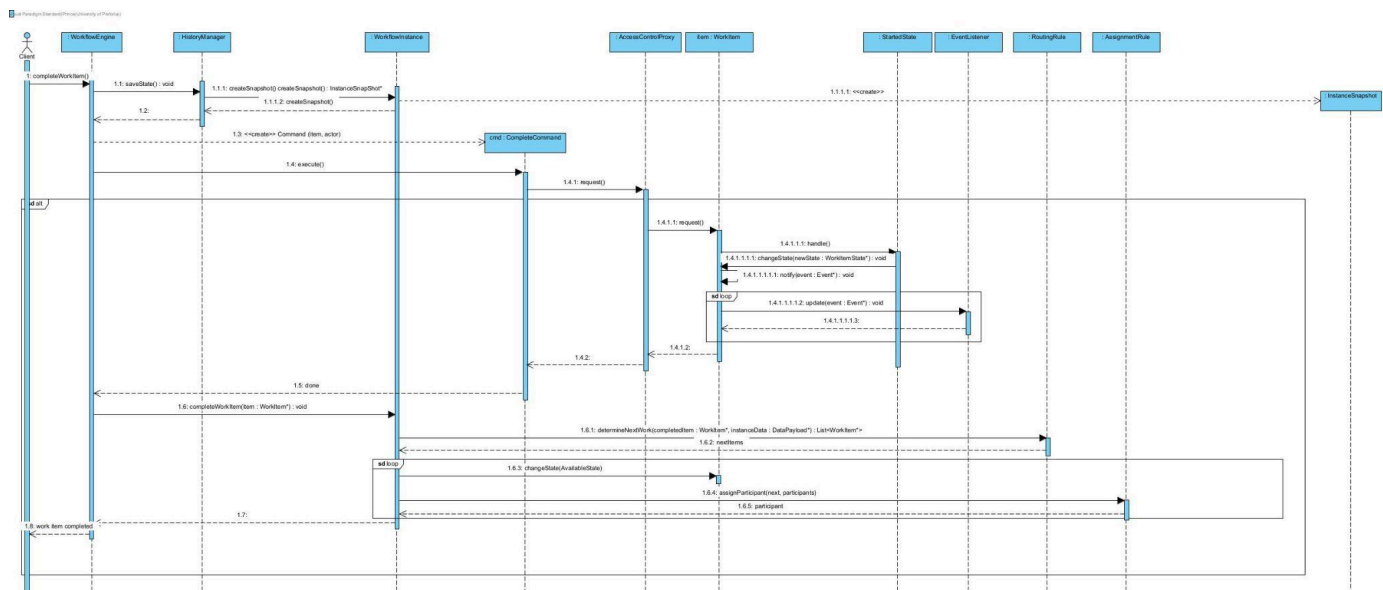
Why the selected pattern is appropriate

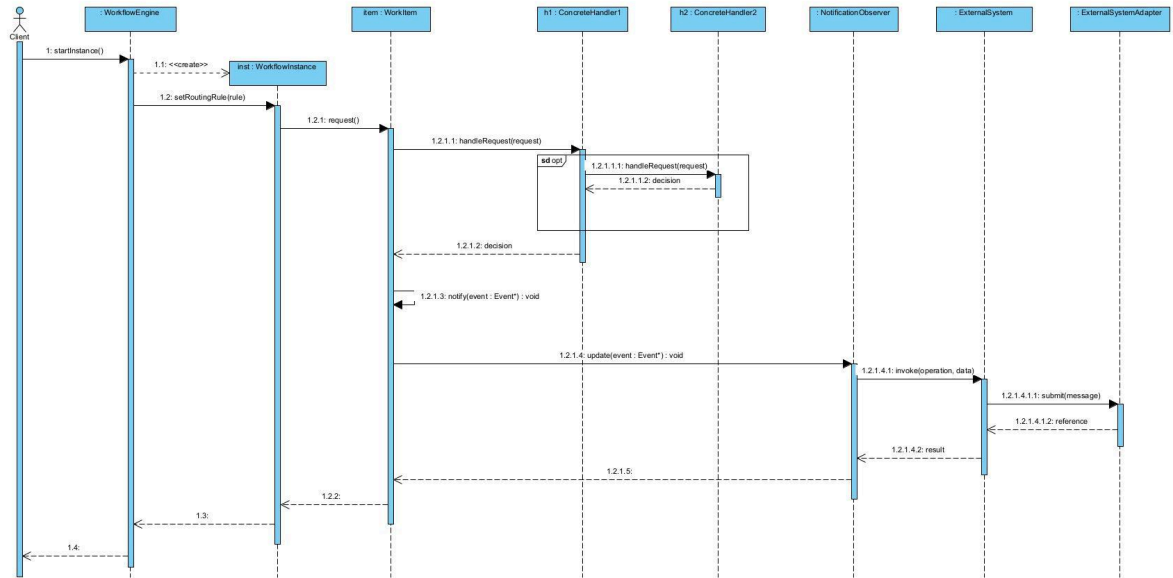
- It prevents the Stage and WorkItem classes from being bloated with navigation logic. The engine can sequentially access specific tasks without needing to understand or parse the underlying complexity of the nested composite tree.

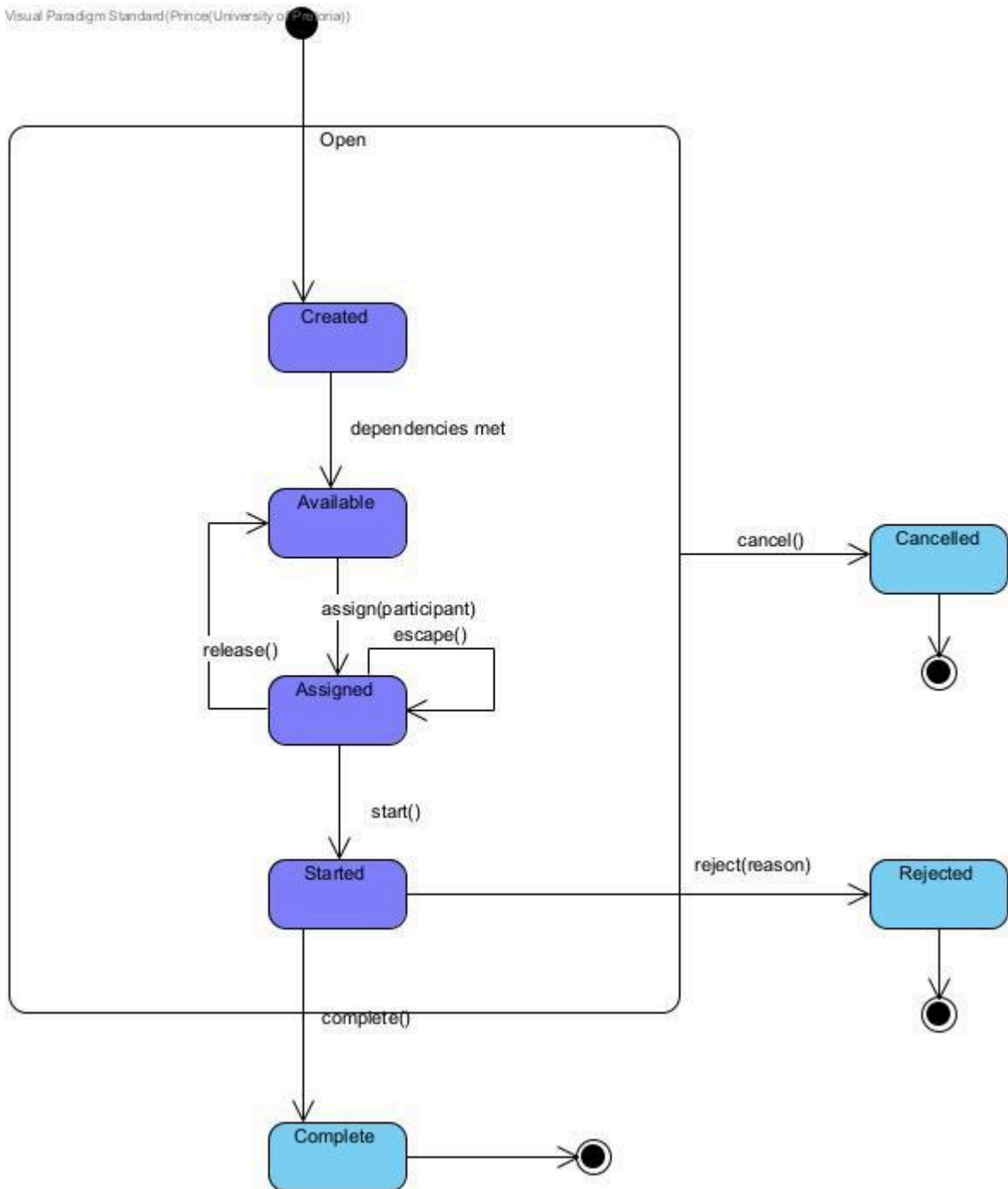
Task 5



Task 6







Task 7

D-01 · 2026-10-03 · Kadin & Frank:

Question: How do we decouple the ArkhamExpress delivery scenario from the workflow engine's core architecture?

Before: The initial design included scenario-specific classes such as *Order*, *Parcel*, *Courier*, and *Hub* directly on the class diagram, with hardcoded states like "Shipped" and "Delivered".

Change: Removed all delivery-specific nouns from the core engine and replaced them with generic abstractions including *WorkflowDefinition*, *WorkflowInstance*, *WorkItem*, and *Participant*. The ArkhamExpress scenario was isolated to the scenario description and one specific activity diagram.

Why: The specification mandates a generic, reusable workflow manager. Baking delivery concepts into the engine prevents it from being used for other processes, such as employee onboarding or insurance claims, without rewriting the classes.

Diagrams: Class Diagram, Generic Activity Diagrams, State Diagram

D-02 · 2026-10-04 · Thakasa:

Question: How do we model the execution of parallel work items within a nested group without hardcoding the routing logic?

Before: Early activity diagram drafts depicted a flat sequence of delivery steps hidden behind basic actions, failing to show how the engine handles nested structures generically.

Change: Designed a generic Activity Diagram demonstrating the engine triggering a *Composite Stage*. Used a fork node to split execution for parallel task assignment, and invoked the Strategy pattern as a called activity (*Evaluate Routing Rules*) to determine the next available tasks.

Why: This satisfies the rubric requirements for swimlanes, a fork/join, and a called activity while proving that the engine can coordinate nested work and delegate assignment without knowing the specific rules of the organization.

Diagrams: Activity Diagram 2

D-03 · 2026-10-06 · Frank & Onalerena:

Question: Where should the system verify if a participant is allowed to perform a specific restricted operation (e.g., a manager cancelling a work item)?

Before: Access control checks were assumed to be written directly inside the *WorkItem* operations or repeated across the engine wherever work was touched.

Change: Introduced the *AccessControlProxy* pattern to wrap *WorkflowComponent* objects (*WorkItem* and *Stage*). The proxy checks an *AccessPolicy* supplied by the *WorkflowDefinition* before forwarding the call to the real object.

Why: This protection proxy prevents the core work classes from being bloated with permission logic and ensures that the core objects and clients remain unchanged, while allowing different organizations to supply their own varying access rules.

Diagrams: Class Diagram, Sequence Diagram 1